Trabalho AV-2: QuickSort

Pivô aleatório (caso médio) e QuickSort Proporcional α : (1 − α)

Sumário

- A. Introdução
  - Descrição do problema
  - Visão geral do programa
- B. Implementação
  - Estruturas de dados
  - Funções do programa
  - Particionamento (Lomuto)
  - Por que Lomuto e não Hoare
  - Versão i: pivô aleatório
  - Versão ii: QuickSort Proporcional
  - Tratamento do StackOverflowError
  - Decisões sobre casos omissos
  - Geração dos vetores
  - Entrada e saída
- C. Análise de complexidade
  - i. Procedimento de particionamento (Lomuto)
  - ii. QuickSort com pivô aleatório (caso médio)
  - iii. QuickSort Proporcional ($\alpha = 1/2$ e $\alpha = 1/4$)
  - Quadro-resumo
- Referências

A. Introdução

Descrição do problema

O problema é **ordenar** um vetor de n inteiros em ordem crescente. Os vetores usados são **permutações aleatórias dos números de 1 até** n, ou seja, cada número aparece exatamente uma vez, em ordem embaralhada.

O algoritmo usado é o **QuickSort**, que funciona por divisão e conquista:

1. escolhe um elemento, o **pivô**;
2. **particiona** o vetor: os menores que o pivô vão para a esquerda, os maiores para a direita, e o pivô fica na sua posição definitiva;
3. ordena do mesmo jeito o lado esquerdo e o lado direito.

O desempenho depende de **quão equilibrados** ficam os dois lados, e isso depende só da escolha do pivô. Por isso foram implementadas duas versões que usam **o mesmo particionamento** e mudam apenas a escolha do pivô:

- **Versão i, QuickSort com pivô aleatório:** o pivô é sorteado.
- **Versão ii, QuickSort Proporcional:** recebe um valor α ($0 < \alpha \leq 1/2$) e escolhe o pivô de forma que todo pedaço de tamanho n seja dividido em αn e $(1-\alpha)n$. Foram usados $\alpha = 1/2$ (partição 50%–50%) e $\alpha = 1/4$ (25%–75%).

Visão geral do programa

Todo o código está em um único arquivo, `QuickSort.java`. O `main` faz, para cada tamanho $n \in \{1\,000;\ 10\,000;\ 100\,000;\ 1\,000\,000\}$:

1. **gera** um vetor com uma permutação aleatória de 1 a n;
2. **ordena** com o QuickSort aleatório;
3. **verifica** se o vetor ficou exatamente $1, 2, \ldots, n$;
4. repete os passos 1 a 3 cinco vezes e **imprime** o tempo médio;
5. faz o mesmo com o QuickSort Proporcional, para $\alpha = 0{,}5$ e $\alpha = 0{,}25$.

Para compilar e executar (Java 8 ou superior):

```
javac QuickSort.java
java QuickSort
```

B. Implementação

Estruturas de dados

O programa usa apenas:

- **vetor de inteiros (`int[]`)**: guarda os números a ordenar. A ordenação é feita no próprio vetor, sem vetor auxiliar. As funções recebem o vetor e dois índices, `ini` e `fim`, que delimitam o **pedaço** (subvetor) `v[ini..fim]` que está sendo tratado;
- **`Random`**: gerador de números aleatórios da biblioteca do Java, usado para sortear o pivô e para embaralhar os vetores.

Não foi necessário criar outros tipos abstratos de dados: o “pedaço” do vetor é representado só pelo par de índices (`ini`, `fim`), e a pilha de recursão do próprio Java guarda os pedaços pendentes.

Funções do programa

Função	O que faz
`particionar(v, ini, fim)`	Particionamento de Lomuto. Usa `v[fim]` como pivô e devolve a posição final `q` do pivô.
`trocar(v, i, j)`	Troca o conteúdo das posições `i` e `j`.
`quickSortAleatorio(v, ini, fim)`	Versão i. Sorteia o pivô, particiona e continua nos dois lados.
`quickSortProporcional(v, ini, fim, alfa)`	Versão ii. Escolhe o pivô que divide o pedaço na proporção $\alpha : (1-\alpha)$.
`gerarVetor(n)`	Gera uma permutação aleatória de 1 a n.
`estaOrdenado(v)`	Confere se o vetor ficou exatamente $1, 2, \ldots, n$.
`main`	Roda os testes e imprime os resultados.

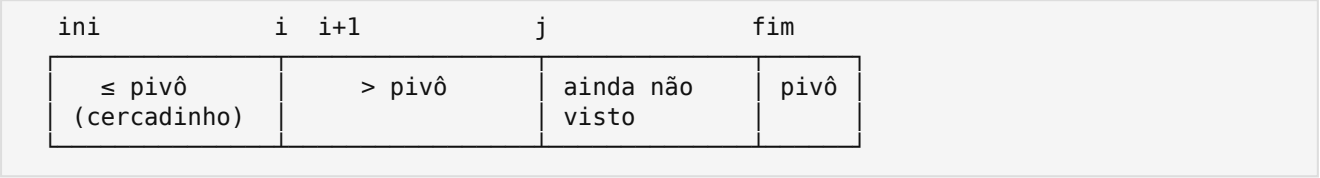
Particionamento (Lomuto)

O pivô é sempre o **último** elemento do pedaço, `v[fim]`. Duas variáveis fazem o trabalho:

- **`i`, o "cercadinho"**: marca o fim da região dos elementos $\leq$ pivô. Começa em `ini - 1` (cercadinho vazio).
- **`j`, o "olheiro"**: percorre o pedaço uma única vez, de `ini` até `fim - 1`. Quando encontra um elemento $\leq$ pivô, o cercadinho cresce uma posição (`i++`) e esse elemento é trocado para dentro dele.

No final, o pivô é trocado com `v[i+1]`, logo depois do cercadinho, que é a sua posição definitiva.

Situação no meio do laço:



Exemplo: `v = [3, 8, 2, 5, 1, 4, 7, 6]`, pivô $= 6$.

j	v[j]	≤ 6?	o que acontece	vetor
0	3	sim	`i = 0`, troca(0,0)	`[3, 8, 2, 5, 1, 4, 7, 6]`
1	8	não	nada	`[3, 8, 2, 5, 1, 4, 7, 6]`
2	2	sim	`i = 1`, troca(1,2)	`[3, 2, 8, 5, 1, 4, 7, 6]`
3	5	sim	`i = 2`, troca(2,3)	`[3, 2, 5, 8, 1, 4, 7, 6]`
4	1	sim	`i = 3`, troca(3,4)	`[3, 2, 5, 1, 8, 4, 7, 6]`
5	4	sim	`i = 4`, troca(4,5)	`[3, 2, 5, 1, 4, 8, 7, 6]`
6	7	não	nada	`[3, 2, 5, 1, 4, 8, 7, 6]`
fim			troca(5,7): pivô no lugar	`[3, 2, 5, 1, 4, 6, 7, 8]`

Resultado: $q = 5$. À esquerda ficam 5 elementos menores que 6 e à direita 2 maiores. Foram feitas $7 = n - 1$ comparações.

Por que Lomuto e não Hoare

O enunciado permite escolher entre Lomuto e Hoare. Foi escolhido o **Lomuto** pelos seguintes motivos:

1. **No Lomuto o pivô termina na posição definitiva.** Depois de particionar, o pedaço fica dividido em `v[ini..q-1]` e `v[q+1..fim]`, com exatamente k e $n - 1 - k$ elementos. É exatamente a forma das recorrências da análise, $T(n) = T(k) + T(n - 1 - k) + cn$.
2. **Isso torna possível a versão Proporcional.** Para forçar a divisão $a : (1 - a)$ basta escolher como pivô o elemento certo. No Hoare o pivô **não** fica necessariamente na posição final, e onde o vetor é dividido depende de como os dados estão arrumados. Assim, seria muito mais difícil garantir o tamanho de cada lado.
3. **É mais simples de entender e de analisar:** um único laço, sempre $n - 1$ comparações.

A desvantagem conhecida do Lomuto é ficar ruim quando há muitos elementos **repetidos**, e ele costuma fazer mais trocas que o Hoare. A primeira não ocorre aqui, porque os vetores não têm repetidos. A segunda só muda a constante, não a complexidade.

Versão i: pivô aleatório

1. Sorteia uma posição `p` entre `ini` e `fim`: `p = ini + rand.nextInt(fim - ini + 1)`.
2. **Troca `v[p]` com `v[fim]`.** Isso é necessário porque o Lomuto sempre usa o **último** elemento como pivô. A troca leva o elemento sorteado para o lugar onde o particionamento espera o pivô.
3. Chama `particionar` e continua nos dois lados (ver o tratamento do StackOverflow, mais abaixo).

Versão ii: QuickSort Proporcional

Para um pedaço com n elementos, o lado esquerdo deve ter

$$k = \lfloor a \cdot (n - 1) \rfloor$$

elementos e o direito $n - 1 - k$ (o -1 é porque o pivô não fica em nenhum dos lados). Para isso, o pivô precisa ser o $(k + 1)$**-ésimo menor** elemento do pedaço.

Como o pivô é encontrado: `valor = ini + 1 + k`. O vetor é uma permutação de 1 a n. Depois de cada particionamento, cada pedaço `v[ini..fim]` contém **exatamente** os valores que vão ocupar essas posições no vetor ordenado. No vetor ordenado, a posição x guarda o valor $x + 1$, porque o Java começa a contar as posições do 0 e os valores começam no 1. Então o pedaço contém os valores $\text{ini} + 1, \text{ini} + 2, \ldots, \text{fim} + 1$. O menor deles é $\text{ini} + 1$, e o $(k + 1)$-ésimo menor é $\text{ini} + 1 + k$.

O programa procura esse valor com uma busca linear (`while (v[p] != valor) p++;`), troca-o com `v[fim]` e particiona.

Exemplo: pedaço inteiro de um vetor de 1 a 8 (`ini = 0`, `fim = 7`, $n = 8$) com $\alpha = 1/4$: $k = \lfloor 0{,}25 \cdot 7 \rfloor = 1$, e o pivô é o valor $0 + 1 + 1 = 2$. Depois de particionar, fica 1 elemento à esquerda (o valor 1) e 6 à direita (os valores 3 a 8).

Tratamento do StackOverflowError

O que causa. No QuickSort "de livro" há duas chamadas recursivas. Se as divisões forem ruins (por exemplo, sempre 0 e $n - 1$), cada chamada abre outra com só um elemento a menos. A pilha chega a n chamadas, e com n na casa das dezenas de milhares o Java estoura a pilha (`StackOverflowError`).

O que foi feito. A recursão é feita **só no lado menor**. O **lado maior** é tratado pelo próprio `while`: em vez de chamar a função de novo, o programa atualiza `ini` ou `fim` e o laço continua.

```
        if (q - ini < fim - q) {                // esquerda é menor
            quickSortAleatorio(v, ini, q - 1); // recursão no menor
            ini = q + 1;                        // o laço continua no maior
        } else {                                // direita é menor
            quickSortAleatorio(v, q + 1, fim);
            fim = q - 1;
        }
```

O `if (q - ini < fim - q)` compara os tamanhos dos dois lados. O lado esquerdo `v[ini..q-1]` tem `q - ini` elementos e o direito `v[q+1..fim]` tem `fim - q`.

Por que funciona. O lado menor tem **no máximo metade** dos elementos. Então cada chamada recursiva recebe no máximo metade do pedaço anterior: $n, n/2, n/4, \ldots$ Depois de $\log_2 n$ chamadas o pedaço tem 1 elemento, então a pilha tem no máximo cerca de $\log_2 n$ chamadas (aproximadamente 20 para $n = 1\,000\,000$). Isso vale **mesmo no pior caso**. O número de operações não muda; muda só a memória usada pela pilha, que cai de $O(n)$ para $O(\log n)$.

```
Sem o tratamento (divisão 0 / n-1)      Com o tratamento
quickSort(n)                            quickSort(n)  → laço: n, n-1, n-2, ..., 1
  quickSort(n-1)                          └ quickSort(0)   (lado menor, volta na hora)
    quickSort(n-2)
      ...          ← n chamadas         ← no máximo 2 chamadas empilhadas
```

Decisões sobre casos omissos

- **Como o pivô do Proporcional é encontrado:** pelo valor `ini + 1 + k` e uma busca linear, como explicado acima. A busca custa $\Theta(n)$, o mesmo que o particionamento, então só aumenta a constante c do termo cn e não muda a complexidade. Isso só funciona porque a entrada é uma permutação de 1 a n, como o enunciado garante. Para entradas quaisquer seria preciso um algoritmo de seleção (por exemplo, a *mediana das medianas*, que também é $\Theta(n)$).

- **Arredondamento de k:** `(int)` corta a parte decimal, ou seja, arredonda para baixo. Como $\alpha \leq 1/2$, o lado esquerdo é sempre o menor ou igual ao direito.
- **Pedaços pequenos:** quando n é pequeno, não dá para dividir exatamente em α. Por exemplo, com $\alpha = 1/4$ e $n = 3$, $k = \lfloor 0{,}5 \rfloor = 0$, e a divisão fica 0 e 2. Isso só acontece quando $\alpha(n-1) < 1$, ou seja, nos pedaços com menos de $1/\alpha + 1$ elementos (menos de 5 para $\alpha = 1/4$). Esses pedaços ficam só nos últimos níveis da recursão e têm tamanho constante, então não mudam a complexidade.
- **α fora do intervalo:** o programa usa só $\alpha = 1/2$ e $\alpha = 1/4$. A partição 90%–10% não foi usada, como pede o enunciado.

Geração dos vetores

`gerarVetor(n)` coloca $1, 2, \ldots, n$ no vetor e embaralha com o algoritmo de **Fisher–Yates**: de trás para frente, troca cada posição i com uma posição sorteada entre 0 e i. Assim não há repetidos, toda permutação tem a mesma chance de sair e o custo é $\Theta(n)$.

Entrada e saída

- **Entrada:** não há entrada do usuário. Os vetores são gerados pelo programa, nos tamanhos $1\,000$, $10\,000$, $100\,000$ e $1\,000\,000$.
- **Saída:** para cada tamanho e cada versão, o **tempo médio** de 5 execuções (em milissegundos, medido com `System.nanoTime()`) e se o vetor ficou **ordenado** (`true`/`false`).

Exemplo de saída (os tempos mudam de máquina para máquina e de execução para execução):

```
n = 1000
  Aleatorio:                       0.28 ms  ordenado = true
  Proporcional (alfa = 0.50):      0.11 ms  ordenado = true
  Proporcional (alfa = 0.25):      0.06 ms  ordenado = true
n = 10000
  Aleatorio:                       0.83 ms  ordenado = true
  Proporcional (alfa = 0.50):      0.88 ms  ordenado = true
  Proporcional (alfa = 0.25):      0.79 ms  ordenado = true
n = 100000
  Aleatorio:                       9.08 ms  ordenado = true
  Proporcional (alfa = 0.50):      9.53 ms  ordenado = true
  Proporcional (alfa = 0.25):      7.81 ms  ordenado = true
n = 1000000
  Aleatorio:                     103.75 ms  ordenado = true
  Proporcional (alfa = 0.50):     98.05 ms  ordenado = true
  Proporcional (alfa = 0.25):     86.24 ms  ordenado = true
```

Ao multiplicar n por 10, o tempo cresce um pouco mais que 10 vezes, como se espera de $n \log n$. Os tempos pequenos ($n = 1\,000$) não são confiáveis, porque o Java ainda está "aquecendo" (compilação JIT) e a medida fica muito curta. Por isso as conclusões sobre complexidade vêm da análise, e não dos tempos.

C. Análise de complexidade

$T(n)$ é o tempo para ordenar um pedaço de n elementos e c é uma constante positiva.

i. Procedimento de particionamento (Lomuto)

Para um pedaço com $n = \text{fim} - \text{ini} + 1$ elementos:

- antes do laço há duas atribuições: custo constante;
- o olheiro `j` vai de `ini` até `fim - 1`, então o laço roda **exatamente** $n - 1$ **vezes**, para qualquer vetor. Cada volta faz uma comparação e, no máximo, um incremento e uma troca: custo constante por volta;
- depois do laço há uma troca e o `return`: custo constante.

Logo, com constantes a, b_1 e b_2:

$$a + b_1(n-1) \;\le\; P(n) \;\le\; a + b_2(n-1).$$

O lado direito mostra que $P(n) = O(n)$. O esquerdo mostra que $P(n) = \Omega(n)$, porque o olheiro **sempre** percorre o pedaço inteiro (não existe saída antecipada). Portanto

$$P(n) = \Theta(n).$$

Somando a escolha do pivô ($\Theta(1)$ no aleatório e $\Theta(n)$ na busca do proporcional), o trabalho feito em cada chamada, fora as chamadas recursivas, é $\Theta(n)$. **Esse é o termo cn que aparece em todas as recorrências a seguir.**

ii. QuickSort com pivô aleatório (caso médio)

Recorrência

O pivô é sorteado entre os n elementos, todos diferentes. Se o pivô sorteado é o $(k+1)$-ésimo menor, o lado esquerdo fica com k elementos e o direito com $n - 1 - k$. Como o sorteio é uniforme, k pode ser $0, 1, \ldots, n-1$, cada um com probabilidade $1/n$. Fazendo a média:

$$T(n) = \frac{1}{n}\sum_{k=0}^{n-1}[T(k) + T(n-1-k)] + cn, \qquad T(0) = T(1) = 0.$$

Por que os dois somatórios viram um só

Separando:

$$\sum_{k=0}^{n-1} T(k) = T(0) + T(1) + \cdots + T(n-1)$$

$$\sum_{k=0}^{n-1} T(n-1-k) = T(n-1) + T(n-2) + \cdots + T(0)$$

São **os mesmos termos em ordem inversa**: quando k vai de 0 a $n-1$, $n-1-k$ vai de $n-1$ a 0. Então as duas somas são iguais:

$$T(n) = \frac{2}{n}\sum_{k=0}^{n-1} T(k) + cn. \qquad (1)$$

Cada tamanho aparece duas vezes: uma como lado esquerdo e outra como lado direito.

Eliminando o somatório

Multiplicando (1) por n:

$$n\,T(n) = 2\sum_{k=0}^{n-1} T(k) + cn^2. \qquad (2)$$

Escrevendo a mesma coisa para $n-1$:

$$(n-1)\,T(n-1) = 2\sum_{k=0}^{n-2} T(k) + c(n-1)^2. \qquad (3)$$

Fazendo (2) $-$ (3), os somatórios se cancelam e sobra só o termo $k = n-1$:

$$n\,T(n) - (n-1)\,T(n-1) = 2\,T(n-1) + c(n^2 - (n-1)^2) = 2\,T(n-1) + c(2n-1).$$

Passando $(n-1)T(n-1)$ para a direita:

$$n\,T(n) = (n+1)\,T(n-1) + c(2n-1).$$

Dividindo tudo por $n(n+1)$:

$$\frac{T(n)}{n+1} = \frac{T(n-1)}{n} + \frac{c(2n-1)}{n(n+1)}. \qquad (4)$$

Telescopando até $T(1)$

Chamando $S(n) = \dfrac{T(n)}{n+1}$, a equação (4) fica $S(n) = S(n-1) + c\,\dfrac{2n-1}{n(n+1)}$. Aplicando de novo para $S(n-1)$, depois $S(n-2)$, e assim por diante, até chegar em $S(1)$:

$$\begin{aligned} S(n) &= S(n-1) + c\,\frac{2n-1}{n(n+1)} \\ &= S(n-2) + c\,\frac{2(n-1)-1}{(n-1)n} + c\,\frac{2n-1}{n(n+1)} \\ &\;\;\vdots \\ &= S(1) + c\sum_{i=2}^{n} \frac{2i-1}{i(i+1)}. \end{aligned}$$

Todos os S intermediários somem; sobram só $S(1)$ e a soma das parcelas. Esse é o "telescópio".

Relação com a série harmônica

Usando frações parciais:

$$\frac{2i-1}{i(i+1)} = \frac{3}{i+1} - \frac{1}{i}$$

(conferindo: $\frac{3i-(i+1)}{i(i+1)} = \frac{2i-1}{i(i+1)}$). Com a **série harmônica** $H_n = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}$:

$$\sum_{i=2}^{n} \frac{3}{i+1} = 3(H_{n+1} - 1 - \tfrac{1}{2}), \qquad \sum_{i=2}^{n} \frac{1}{i} = H_n - 1.$$

Como $H_{n+1} = H_n + \frac{1}{n+1}$:

$$\sum_{i=2}^{n} \frac{2i-1}{i(i+1)} = 2H_n + \frac{3}{n+1} - \frac{7}{2}.$$

Voltando para $T(n) = (n+1)S(n)$, com $S(1) = T(1)/2 = 0$:

$$T(n) = c\,(n+1)(2H_n + \frac{3}{n+1} - \frac{7}{2}) = 2c\,(n+1)H_n + O(n).$$

A série harmônica cresce como o logaritmo: $\ln(n+1) \le H_n \le 1 + \ln n$, ou seja, $H_n \approx \ln n$. Então

$$T(n) \approx 2c\,n \ln n \approx 1{,}39\,c\,n \log_2 n \qquad \Rightarrow \qquad T(n) = \Theta(n \log n) \text{ no caso médio}$$

Por que vale para qualquer sequência de entrada

A aleatoriedade está **no algoritmo** (no sorteio do pivô), e não nos dados. Para **qualquer** vetor de entrada, seja ordenado, invertido ou qualquer outro, o pivô sorteado num pedaço de tamanho m tem a mesma chance $1/m$ de ser o 1º, o 2º, …, o m-ésimo menor. Então a recorrência acima vale igual para todas as entradas, e nenhum vetor consegue forçar o algoritmo a ir mal. É como sortear o juiz: ninguém consegue escolher um jogo “armado”. Isso é diferente de usar sempre o último elemento como pivô, em que um vetor já ordenado sempre dá o pior caso.

O pior caso ainda pode ocorrer?

Pode, mas é muito improvável. Nada impede o sorteio de escolher, por azar, sempre o menor ou o maior elemento. Nesse caso as divisões são 0 e $n-1$ e o tempo é $\Theta(n^2)$ (ver o fim do item iii). Num pedaço de tamanho m a chance de sortear um extremo é $2/m$. Para isso acontecer em todos os níveis, a probabilidade é

$$\frac{2}{n} \cdot \frac{2}{n-1} \cdots \frac{2}{2} = \frac{2^{n-1}}{n!},$$

que já é cerca de $0{,}00014$ para $n = 10$ e praticamente zero para n grande. Resumo: **melhor caso e caso médio $\Theta(n \log n)$, pior caso $\Theta(n^2)$, mas raríssimo e sem depender da entrada.**

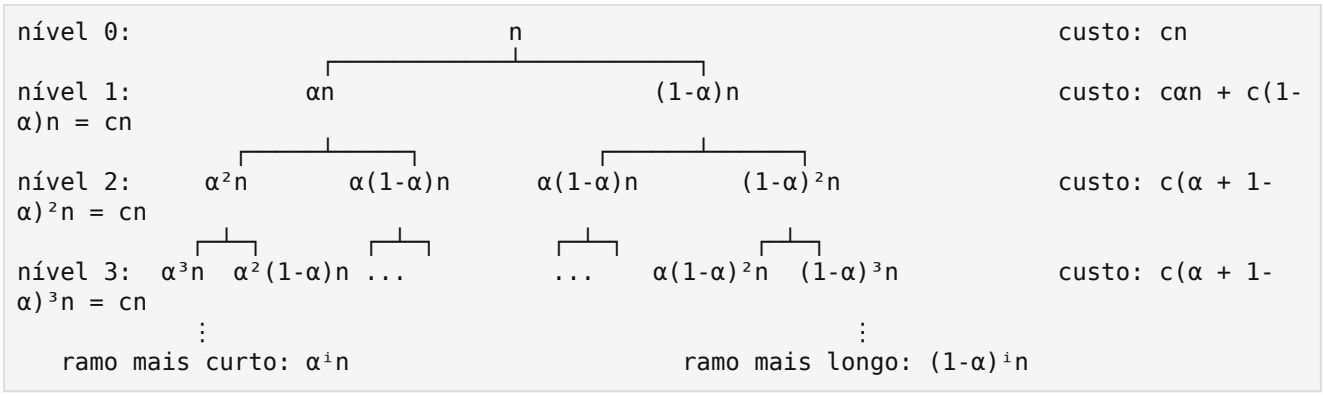
iii. QuickSort Proporcional ($\alpha = 1/2$ e $\alpha = 1/4$)

Todo pedaço de tamanho n é dividido em αn e $(1-\alpha)n$ (desprezando arredondamentos e o pivô, que não mudam a ordem de grandeza):

$$T(n) = T(\alpha n) + T((1-\alpha)n) + cn.$$

Árvore de recursão (caso geral)

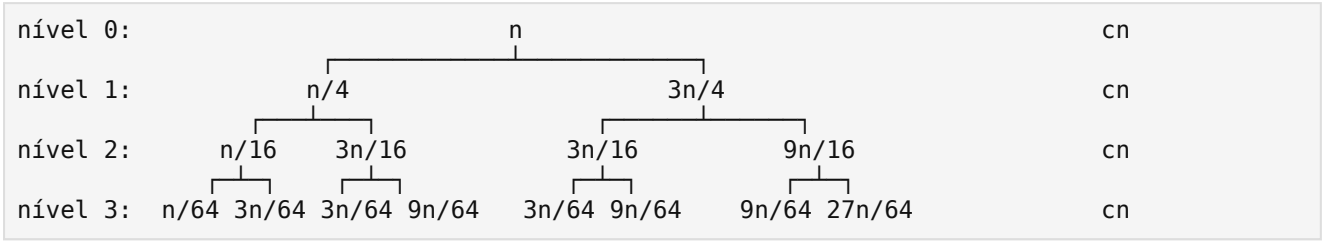
Cada nó mostra o tamanho do pedaço; o custo do nó é c vezes esse tamanho.



No nível 3 há um pedaço de tamanho $a^3 n$, três de $a^2(1-a)n$, três de $a(1-a)^2 n$ e um de $(1-a)^3 n$. Esses são os termos de $(a + (1-a))^3 = 1$, então a soma é n. Isso faz sentido, porque cada nível só reparte os mesmos elementos em pedaços menores. Portanto:

- **todo nível completo custa** cn;
- os níveis de baixo, depois que o ramo mais curto acabou, estão incompletos e custam **no máximo** cn.

Exemplo com $a = 1/4$



Conferindo o nível 3: $\frac{1+3+3+9+3+9+9+27}{64} = \frac{64}{64} = 1$, ou seja, soma n.

Exemplo com $a = 1/2$



Com $a = 1/2$ **os dois lados são iguais**: todos os pedaços de um nível têm o mesmo tamanho, a árvore é perfeitamente equilibrada e todos os ramos têm o mesmo comprimento. Logo $h_{\min} = h_{\max}$.

$h_{\min}$ e $h_{\max}$

Ramo mais curto: segue sempre o lado menor, $a^i n$, e termina quando o pedaço chega a 1:

$$n \cdot a^{h_{\min}} = 1 \Rightarrow \left(\tfrac{1}{a}\right)^{h_{\min}} = n \Rightarrow h_{\min} = \log_{1/a} n.$$

Ramo mais longo: segue sempre o lado maior, $(1-a)^i n$:

$$n \cdot (1-\alpha)^{h_{\max}} = 1 \Rightarrow h_{\max} = \log_{1/(1-\alpha)} n.$$

- $\alpha = 1/2$: $h_{\min} = h_{\max} = \log_2 n$.
- $\alpha = 1/4$: $h_{\min} = \log_4 n = \frac{1}{2}\log_2 n$ e $h_{\max} = \log_{4/3} n$.

Limite inferior e superior

Inferior. Do nível 0 até o nível $h_{\min}$ todos os níveis estão completos e cada um custa cn. Ignorando o resto, que só aumentaria o custo:

$$T(n) \geq cn \cdot h_{\min} = cn \log_{1/\alpha} n \Rightarrow T(n) = \Omega(n \log n).$$

Superior. A árvore tem no máximo $h_{\max} + 1$ níveis, cada um com custo no máximo cn:

$$T(n) \leq cn(h_{\max} + 1) = cn \log_{1/(1-\alpha)} n + cn \Rightarrow T(n) = O(n \log n).$$

Em resumo: $cn \cdot h_{\min} \leq T(n) \leq cn(h_{\max} + 1)$.

$h_{\max}$ na forma $k \cdot \log_2 n$

Pela mudança de base, $\log_b n = \dfrac{\log_2 n}{\log_2 b}$:

$$h_{\max} = \log_{1/(1-\alpha)} n = \frac{1}{\log_2 \frac{1}{1-\alpha}} \cdot \log_2 n = k \cdot \log_2 n, \qquad k = \frac{1}{\log_2 \frac{1}{1-\alpha}}.$$

- $\alpha = 1/2$: $k = \dfrac{1}{\log_2 2} = 1$, então $h_{\max} = \log_2 n$.
- $\alpha = 1/4$: $k = \dfrac{1}{\log_2 (4/3)} = \dfrac{1}{2 - \log_2 3} \approx \dfrac{1}{0{,}415} \approx 2{,}41$, então $h_{\max} \approx 2{,}41 \log_2 n$.

Por exemplo, para $n = 1\,000\,000$ ($\log_2 n \approx 20$): com $\alpha = 1/2$ a árvore tem cerca de 20 níveis. Com $\alpha = 1/4$, o ramo mais curto tem cerca de 10 níveis e o mais longo cerca de 48.

Conclusão

- $\alpha = 1/2$: $h_{\min} = h_{\max} = \log_2 n$, todos os níveis completos, então $T(n) = cn \log_2 n = \Theta(n \log n)$. É o melhor caso possível do QuickSort.
- $\alpha = 1/4$: $\frac{c}{2} n \log_2 n \leq T(n) \leq 2{,}41\, cn \log_2 n + cn$, então $T(n) = \Theta(n \log n)$, com uma constante maior que a de $\alpha = 1/2$.

Para **qualquer α constante** com $0 < \alpha \leq 1/2$, os dois limites são (constante) $\times\, n \log n$. Então

$$T(n) = \Theta(n \log n).$$

O α só muda a **constante** k: quanto menor o α, mais alta e “torta” a árvore. Mesmo uma divisão 1%–99% continua $\Theta(n \log n)$, com $k \approx 69$. Basta que a divisão seja **proporcional**; não precisa ser equilibrada.

E se a partição não for proporcional?

Se a divisão for sempre **0 e** $n-1$ (pivô sempre o menor ou o maior), o lado menor deixa de ser uma fração de n e passa a ter tamanho constante:

$$T(n) = T(n-1) + T(0) + cn = T(n-1) + cn.$$

Abrindo:

$$T(n) = cn + c(n-1) + c(n-2) + \cdots + c \cdot 2 = c\left(\frac{n(n+1)}{2} - 1\right) = \Theta(n^2).$$

A árvore vira uma **linha** de altura $n-1$, e o custo dos níveis diminui de um em um (cn, $c(n-1)$, …) em vez de ficar cn em todos. O que muda:

- a altura passa de $\Theta(\log n)$ para $\Theta(n)$, porque o tamanho diminui **subtraindo 1** a cada nível, em vez de ser **dividido** por um fator constante;
- o tempo passa de $\Theta(n \log n)$ para $\Theta(n^2)$;
- sem o tratamento da seção B, a pilha teria n chamadas e daria `StackOverflowError`.

O mesmo acontece com qualquer divisão em que o lado menor tenha tamanho constante (1 e $n-2$, 10 e $n-11$, …).

Quadro-resumo

Versão	Recorrência	Melhor caso	Caso médio	Pior caso	Pilha (com o tratamento)
Particionamento (Lomuto)	—	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$O(1)$
QuickSort pivô aleatório	$T(n) = \frac{2}{n}\sum_{k=0}^{n-1} T(k) + cn$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$ (muito improvável)	$O(\log n)$
Proporcional $\alpha = 1/2$	$T(n) = 2T(n/2) + cn$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(\log n)$
Proporcional $\alpha = 1/4$	$T(n) = T(n/4) + T(3n/4) + cn$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(\log n)$
Proporcional, α constante	$T(n) = T(\alpha n) + T((1-\alpha)n) + cn$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(\log n)$
Partição não proporcional (0 e $n-1$)	$T(n) = T(n-1) + cn$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$ (sem o tratamento: $O(n)$)

α	$h_{\min}$	$h_{\max} = k \log_2 n$	k
1/2	$\log_2 n$	$\log_2 n$	1
1/4	$\frac{1}{2}\log_2 n$	$\log_{4/3} n$	≈2,41

Na versão Proporcional, a divisão é imposta pelo algoritmo. Por isso, para um α fixo, melhor, médio e pior caso são iguais.

Referências

- CORMEN, T. H. et al. *Algoritmos: teoria e prática*. 3. ed. Rio de Janeiro: Elsevier, 2012. Cap. 7 (Quicksort) e cap. 4 (árvore de recursão).
- ZIVIANI, N. *Projeto de Algoritmos com implementações em Java e C++*. São Paulo: Thomson, 2007.